PROJECT REPORT

# ER-TRIAGE SYSTEM

Emergency Room Patient Triage & Queue Management System

Java | Java Swing GUI | Object-Oriented Design

| | |
|---|---|
| **Student Name** | Triparna Das |
| **Student ID** | 12024052017038 |
| **Language** | Java 11+ | No external libraries |
| **Project Scale** | Small / Medium |
| **Assessment** | Individual Project |

## 1.Project Overview

### 1.1 Project Title

ER-Triage — Emergency Room Patient Triage & Queue Management System

### 1.2 Problem Statement

Emergency rooms face life-or-death decisions every minute. Without a structured triage system, staff risk treating minor injuries while critical patients deteriorate in the waiting room. Most manual systems rely on staff judgment, leading to inconsistent prioritisation and delayed care.

This system digitises the 5-level Emergency Severity Index (ESI) protocol, automatically assessing incoming patients — General, Trauma, and Cardiac — based on clinical vitals, assigning a priority level, and managing a sorted queue so the most critical patients are always seen first. A full Java Swing GUI allows ER staff to admit, track, and discharge patients, while a background monitoring thread continuously alerts when critical patients exceed safe wait times.

### 1.3 Target User

ER nurses and triage coordinators in hospital emergency departments who need a fast, structured system to manage patient intake and priority ordering under time pressure.

### 1.4 Core Features

| |
|---|
| • Automated ESI triage assessment — clinical rules assign levels 1–5 from patient vitals |
| • Priority Queue Management — patients sorted automatically by severity, not arrival time |
| • Three patient types — General, Trauma, and Cardiac with type-specific assessment logic |

- Background monitoring thread — alerts staff when critical patients exceed safe wait times
- Patient lookup and discharge — search by ID, update status, remove from queue
- File logging and CSV export — all activity logged to records/er_log.txt
- Full Java Swing GUI — dark-themed, multi-panel interface with sidebar navigation
- Input validation — all clinical inputs validated with descriptive error messages

# 2. System Architecture

## 2.1 Layered Architecture

The system follows a strict layered architecture where each layer has a single responsibility. Higher layers call into lower layers but never the reverse, ensuring clean separation of concerns.

| Layer | Classes | Responsibility |
|---|---|---|
| UI Layer | ERMainWindow, ERComponents, ERTh | emeSwing GUI — panels, forms, tables, navigation, styling |
| Service Layer | TriageService, FileService | Business logic — queue management, patient registry, file I/ |
| Model Layer | Patient (abstract), GeneralPatient, TraumaPatient, CardiacPatient, TriageL | Domain data and triage assessment logic vel |
| Support | InputValidator, ERMonitorThread, 3 Custom Exceptions | Validation, background monitoring, error handling |

## 2.2 Folder Structure

The project uses a flat package structure — each subfolder under src/ corresponds directly to a Java package, making navigation and import statements simple and consistent.

```
ER-Triage/ src/ Main.java -- entry point, wires services, launches GUI model/ -- Patient (abstract), TriageLevel, 3 subclasses service/ -- TriageService, FileService exception/ -- 3 custom exception classes util/ -- InputValidator (static utility) thread/ -- ERMonitorThread (implements Runnable) ui/ -- ERMainWindow, ERComponents, ERTheme records/ -- er_log.txt and CSV exports saved here at runtime compile.bat -- Windows compile script run.bat -- Windows run script
```

# 3. OOP Concepts Implemented

## 3.1 Abstraction

The **Patient** class is declared abstract, forcing all subclasses to implement *assessTriageLevel()* and *getClinicalSummary()*. This enforces the rule at compile time: any new patient type must provide its own triage assessment logic, preventing silent bugs. The abstract class also holds shared state (name, age, patientId, arrivalTime, status) that all subtypes inherit without code duplication.

| Abstract Method | Purpose |
|---|---|
| assessTriageLevel() | Each patient type implements its own clinical rules to determine ESI level 1–5 |
| getClinicalSummary() | Returns a one-line summary of the patient's key clinical measurements |

## 3.2 Encapsulation

All fields across every class are declared **private**. Access is only possible through getter and setter methods. The **InputValidator** utility class encapsulates all clinical validation rules behind static methods, hiding the validation logic entirely from the UI layer. This means the internal state of a Patient can never be corrupted by external code.

## 3.3 Inheritance

Three concrete patient types extend the abstract Patient class, inheriting all shared fields (patientId, name, age, bloodType, arrivalTime, status) and implementing only what is unique to their type. This eliminates code duplication and ensures consistent behaviour.

| Subclass | Unique Fields | Triage Rule Summary |
|---|---|---|
| GeneralPatient | chiefComplaint, painScale, temperature, respiratoryRate | RR<8/>30 or temp>40.5 → ESI-1 pain≥8 or temp>39.5 → ESI-2 |
| TraumaPatient | injuryType, glasgowComaScale, isConscious, hasInternalBleeding | Unconscious or GCS≤8 → ESI-1 bleeding or GCS≤12 → ESI-2 |
| CardiacPatient | heartRate, systolicBP, chestPain, stElevation | STEMI or BP<80 or HR>150 → ESI-1 chestPain + HR>120 → ESI-2 |

## 3.4 Polymorphism

The **PriorityQueue<Patient>** in TriageService holds all three patient types simultaneously. When the queue iterates and calls *getClinicalSummary()* or *assessTriageLevel()*, Java automatically dispatches to the correct overridden version in the subclass — runtime polymorphism. The *compareTo()* method in Patient enables the PriorityQueue to sort patients by ESI level without knowing which specific subtype they are.

## 3.5 Exception Handling

Three custom exception classes demonstrate both checked and unchecked exception design. Checked exceptions force the caller (ERMainWindow) to handle them explicitly with try-catch, preventing silent failures. The unchecked exception represents a programming error.

| Exception Class | Type | When Thrown |
|---|---|---|
| PatientNotFoundException | Checked | Patient ID not found in registry during lookup or discharge |
| DuplicatePatientException | Checked | Same patient ID attempted to be admitted twice |
| InvalidTriageDataException | Unchecked | Clinical input fails validation (e.g. pain scale 15, age -1) |

### 3.6 Collections

Three Java collections are used together in TriageService, each chosen for a specific performance characteristic that matches its use case in the system.

| Collection | Performance | Purpose |
|---|---|---|
| PriorityQueue<Patient> | O(log n) insert<br>O(1) peek | Auto-sorted waiting queue via compareTo() |
| HashMap<String,Patient> | O(1) lookup | Patient registry — instant ID-based access |
| ArrayList<Patient> | O(1) append | Full history including discharged patients for CSV export |

### 3.7 File Handling

FileService manages all file I/O using Java's standard library. Activity logging uses **FileWriter** with **BufferedWriter** in append mode inside try-with-resources blocks, ensuring files are always closed. CSV export uses **PrintWriter** for formatted output. The records/ directory is automatically created on startup using NIO's Files.createDirectories().

### 3.8 Threads

ERMonitorThread **implements Runnable** (preferred over extending Thread to preserve single inheritance flexibility). A **volatile boolean** flag ensures the stop() signal from the main thread is immediately visible to the monitor thread — without volatile, the JVM may cache the flag. The thread is set as a **daemon thread** in Main.java, so it automatically terminates when the application exits without requiring manual cleanup.

## 4. GUI Design

The interface is built entirely with Java Swing. It uses a dark professional colour scheme (background #0A0E14, accent #00D4FF) with a fixed sidebar navigation, header stats bar, and a CardLayout content area. Three UI classes divide responsibilities cleanly.

| UI Class | Responsibility |
|---|---|
| ERTheme.java | Central colour palette (15 colours) and font constants. Changing the visual style only requires editin |
| ERComponents.java | Factory methods for styled Swing widgets — buttons, panels, tables, inputs, scroll panes. Used throu |
| ERMainWindow.java | Main JFrame. Contains all panels, forms, tables, event handlers, and the admit/discharge/lookup bus |

### 4.1 GUI Panels

| Panel | Functionality |
|---|---|
| Admit Patient | Type selector (General / Trauma / Cardiac). Each form collects clinical data. Triage level auto-assigned |
| Waiting Queue | Live JTable sorted by ESI priority. Each row has a Discharge button (custom CellEditor). 'Call Next Pati |
| ER Dashboard | Five colour-coded stat cards (one per ESI level) with patient counts. Full patient history JTable showing |
| Lookup Patient | Search by ID using JTextField. Shows a full detail panel with patient data and a discharge option if statu |
| Activity Log | Right-side JTextArea with timestamped entries. Categories: ADMIT, CALL, DISC, ALERT, SYS. Blink |

## 5. Class Descriptions

| Class | Package | Description |
|---|---|---|
| Patient | model | Abstract base class. Shared state and compareTo() for PriorityQueue orderin |
| TriageLevel | model | Enum: 5 ESI levels, each storing priority number and description. |
| GeneralPatient | model | Extends Patient. Triage rules: pain scale, temperature, respiratory rate. |
| TraumaPatient | model | Extends Patient. Triage rules: Glasgow Coma Scale and consciousness. |
| CardiacPatient | model | Extends Patient. Triage rules: heart rate, blood pressure, ST-elevation. |
| TriageService | service | Core service. PriorityQueue + HashMap + ArrayList. Single source of patien |
| FileService | service | Activity logging to er_log.txt and CSV export using BufferedWriter and Prin |
| PatientNotFoundException | exception | Checked exception — patient ID not found in registry. |
| DuplicatePatientException | exception | Checked exception — duplicate patient admission attempt. |
| InvalidTriageDataException | exception | Unchecked (RuntimeException) — clinical input out of valid range. |
| InputValidator | util | Static utility class. Validates all clinical input ranges before service layer. |
| ERMonitorThread | thread | Runnable daemon thread. Checks every 5s for patients exceeding safe wait ti |
| ERMainWindow | ui | Main JFrame. All panels, forms, tables, and event handlers. |
| ERComponents | ui | Factory for dark-theme Swing widgets — used by ERMainWindow through |
| ERTheme | ui | Colour palette and font constants for the entire GUI. |
| Main | (root) | Entry point. Wires services, starts daemon thread, launches ERMainWindow |

# 6. Design Decisions & Justification

**Why abstract Patient instead of an interface?**

Patient needs shared *state* — fields like name, age, patientId, arrivalTime. An interface cannot hold instance state. An abstract class defines shared fields once and is inherited, while abstract methods still enforce the contract on all subtypes.

**Why PriorityQueue instead of ArrayList with sorting?**

ArrayList.sort() is O(n log n) and must be called every admission. PriorityQueue maintains sorted order automatically at O(log n) insertion with O(1) access to the highest-priority patient. In a busy ER, patients are admitted rapidly, so PriorityQueue is the correct structure.

**Why a separate HashMap registry alongside the PriorityQueue?**

PriorityQueue does not support key-based lookup — finding a patient by ID requires O(n) iteration. The HashMap provides O(1) lookup, essential for frequent discharge and detail-view operations. The cost is minimal extra memory to store references.

**Why daemon thread for ERMonitorThread?**

A daemon thread automatically terminates when the JVM's last non-daemon thread exits. This means the monitor stops cleanly when the user closes the window — no explicit shutdown logic is needed.

**Why volatile for the running flag?**

In multi-threaded code, the JVM may cache variable values per-thread for performance. Without volatile, the main thread calling stop() might update its local cache without the monitor thread ever seeing the change. Volatile forces all reads/writes to main memory, guaranteeing cross-thread visibility without synchronisation overhead.

**Why checked exceptions for PatientNotFoundException and DuplicatePatientException?** These are expected, recoverable error conditions in normal operation. Checked exceptions force every call site to handle them explicitly, preventing silent failures. InvalidTriageDataException is unchecked because it represents a programming error — if input validation is applied correctly, it should never reach the service layer.

# 7. How to Compile and Run

## 7.1 Requirements

- Java 11 or higher (Java 17 recommended)
- No external libraries, Maven, or Gradle required

## 7.2 Steps (Windows)

1. Extract the ER-Triage folder to your desktop or chosen location
2. Open the ER-Triage folder in VS Code: File → Open Folder
3. Open the terminal: View → Terminal (or Ctrl + `)
4. Compile: type compile.bat and press Enter
5. Run: type run.bat and press Enter
6. The dark-themed Swing GUI window will open automatically

### 7.3 Manual Compile Command

```
mkdir out
javac -sourcepath src -d out src/Main.java src/model/*.java ^
src/service/*.java src/exception/*.java ^
src/util/*.java src/thread/*.java src/ui/*.java
java -cp out Main
```

### 7.4 Output Files

| File | Contents |
|---|---|
| records/er_log.txt | Timestamped log of all admissions, calls, discharges, and alerts |
| records/patients_export_YYYY-MM-DD.csv | Full patient data export with all fields for the current session |

## 8. ESI Triage Level Reference

The system implements the 5-level Emergency Severity Index (ESI) protocol used in real hospital emergency departments. Lower numbers indicate greater urgency.

| ESI | Name | Max Wait | Clinical Trigger Example |
|---|---|---|---|
| 1 | Resuscitation | Immediate | STEMI, GCS ≤8, RR <8 or >30, BP <80 mmHg |
| 2 | Emergent | 5 minutes | Pain ≥8, temp >39.5°C, internal bleeding, HR >150 |
| 3 | Urgent | 30 minutes | Pain 5–7, GCS 13–14, chest pain alone |
| 4 | Less Urgent | 60 minutes | Pain 3–4, GCS 15 with minor injury |
| 5 | Non-Urgent | 120 minutes | Pain 0–2, routine visit, all vitals normal |

# 9. Summary

The ER-Triage System demonstrates disciplined object-oriented design applied to a real-world problem. Every architectural decision was made for a specific technical reason. The result is a system that is modular, extensible, and fully defensible at every design choice

| OOP Concept | Implementation |
|---|---|
| Abstraction | Patient abstract class with 2 abstract methods enforced at compile time |
| Encapsulation | All fields private; InputValidator hides clinical validation logic |
| Inheritance | 3 subclasses (General/Trauma/Cardiac) extend Patient |
| Polymorphism | PriorityQueue<Patient> holds mixed types; runtime method dispatch |
| Exception Handling | 3 custom exceptions: 2 checked, 1 unchecked |
| Collections | PriorityQueue + HashMap + ArrayList — each for a specific purpose |
| File Handling | FileService — BufferedWriter log + PrintWriter CSV export |
| Threads | ERMonitorThread implements Runnable; volatile flag; daemon thread |
| Swing GUI | Full dark-themed interface: CardLayout, custom buttons, live tables |

ER-Triage System | Java OOP Project Report | All rights reserved